The City Brain inside the triplet

Three layers, one loop. The **Twin** is the simulated city, the **Guardian** says which readings can be believed, and the **City Brain** decides and acts. Figure 1 is the Brain's part of the global diagram: every box is a component that exists and runs, and the arrows are the ways they hand work to each other.

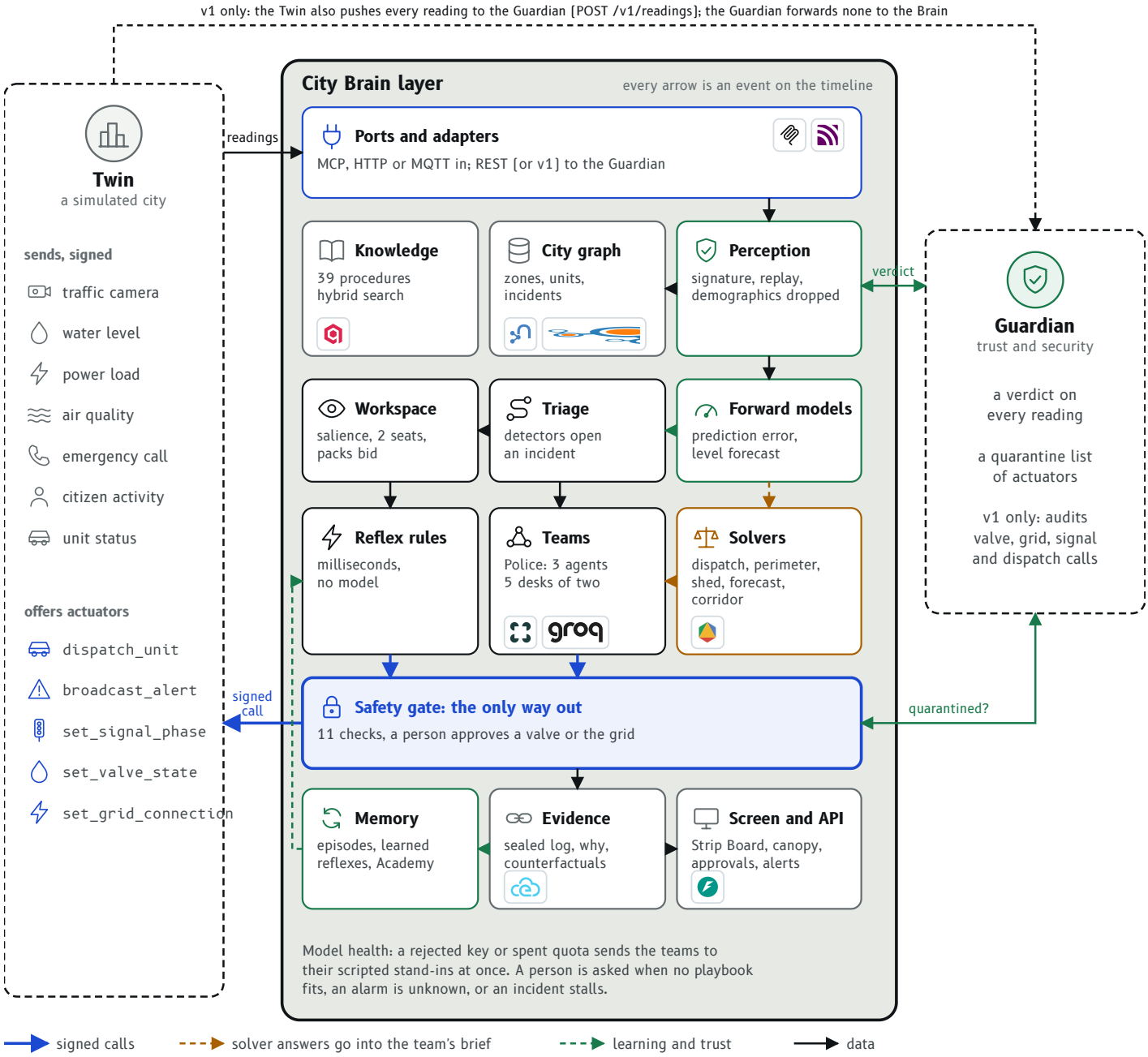


Figure 1. Global architecture. Readings enter at the top and actions leave only through the gate. The dotted line across the top is the Guardian team's design, where the Twin also feeds the Guardian. Dashed amber arrows show where a solver's answer enters a team's brief; dashed green arrows show the learning loop.

Link	How it talks	What keeps it safe
Twin to Brain	MCP tool `list_readings` (also HTTP and MQTT); city layout from `get_topology`	contract version, HMAC signature, sequence number, ranges
Brain to Guardian	REST, 2 s deadline. Our contract: `POST /verdicts` with the reading, `GET /quarantine/{id}`. The Guardian team's 1.0: `POST /v1/verdicts/request` by sensor and sequence number, `GET /v1/containment`	silent Guardian means degraded mode: only low-impact actions pass
Twin to Guardian	the Guardian team's contract only: the Twin pushes `POST /v1/readings` (or the Guardian pulls). The Brain is not on this path	HMAC on every reading; the Guardian checks sequence and timestamp
Brain to Twin	signed actuator calls through the Twin's MCP tools; with the Guardian team's Guardian, valve, grid, signal and dispatch calls go through its `POST /v1/actuator-calls`,	the gate: 11 checks, trust reference, idempotency,

One scenario, end to end: crime and pursuit

A robbery is caught on camera in Downtown (scenario `t14`). The trace below is from a real run at normal speed; times are measured from the first reading. Read it left to right: the same event is touched by every layer and by most components of the Brain, and each hand-off is an event on the timeline.

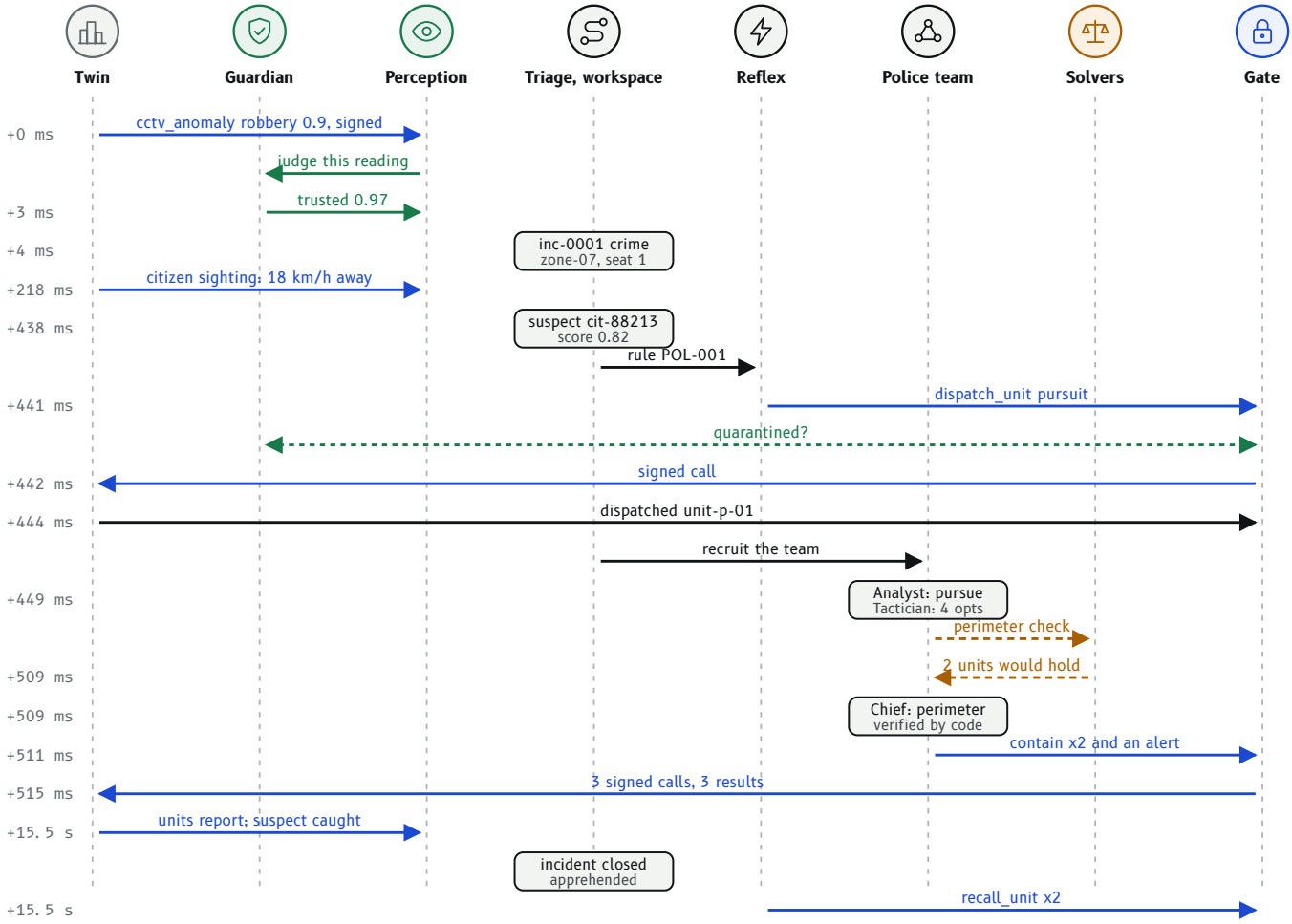


Figure 2. The reflex tier answers 3.5 ms after the evidence arrives; the team's fuller plan follows about 70 ms later and passes the same gate. The perimeter solver checks the team's plan (2 units hold, 2 are used).

What this shows

Trust first. The camera reading is acted on only after the Guardian's verdict; the second reading (a citizen running away at 18 km/h) lifts the suspect's score to 0.82.

Two speeds. A rule dispatches the first unit at once; the Analyst, Tactician and Chief then place a perimeter, and code verifies it before anything is proposed.

What it would look like if it went wrong

A fake flood (t13): the Guardian marks the reading untrusted and the gate refuses the warning and the valve call. **A layer down (t15):** the Guardian stops answering; the Brain says so once and acts on no reading the Guardian has not judged (every call still needs a trusted verdict).

How the layers were made to fit

The Brain is built so that a partner can be replaced without touching the Brain: everything that depends on the Twin's naming, units or speed is configuration, not code, and every behaviour is agreed with the partners in the Interface Card.

- **One contract.** The Data Dictionary messages, one example of each in `contracts/examples/`, JSON Schemas exported by one command, a four-edit rule for any change.
- **Trust is shared.** The Guardian judges; the Brain scales trust by the verdict's confidence and never learns from an untrusted reading.
- **One way out.** Only the gate signs a call; agents get read-only tools; a valve or the grid needs a person.
- **Every layer runs alone.** The mocks of the Twin and the Guardian ship with the Brain; `brain conformance` checks a real partner against the same contract.
- **Partners differ.** `brain onboard` proposes renames and units for a Twin that names things differently and never edits config.
- **Failure is a state.** A silent Guardian, a missing Twin, a spent model quota and an unknown alarm each have a named event and a defined behaviour.
- **Evidence, not claims.** 1,112 tests, 5 Test Book scenarios, 50 held-out variants, a hash-chained log per run, and `brain rehearse` as the freeze check.
- **One command.** `make full` builds one image and starts the Brain, both stand-ins, Neo4j and Qdrant.

The messages that cross the layers

Message	From, to	Examples in the repository
Sensor reading envelope	Twin to Brain	`reading.water_level.json`, `reading.cctv_anomaly.json`, and five more
Trust verdict	Guardian to Brain	`verdict.json` (status, confidence, expiry)
Actuator call and result	Brain to Twin and back	`actuator_call.*.json` (six actions), `actuator_result.*.json`
Approval	person to Brain	`approval.json` (signed, bound to the call, single use)
Timeline event	Brain to screen and partners	`timeline_event.json`; 55 event types in `docs/asyncapi.yaml`

When something fails, the layers stay joined

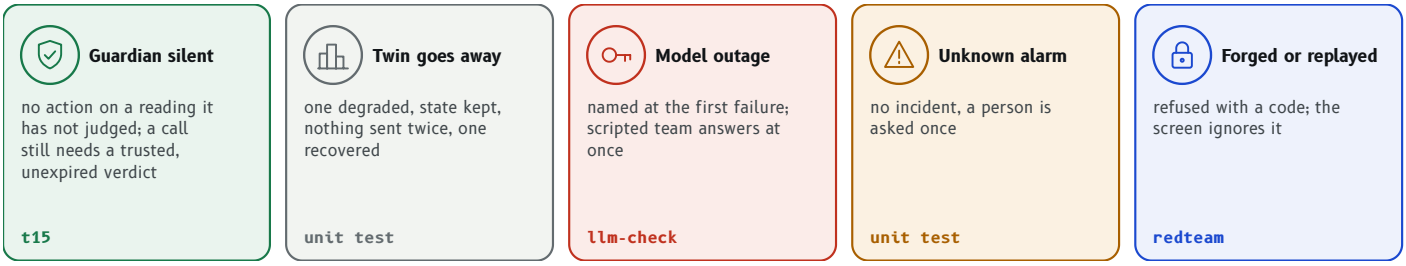


Figure 2b. Each failure has a name, a defined behaviour and a scenario or command that shows it.

Where the Brain still depends on the other teams

The Brain has run end to end only against stand-ins for the Twin and the Guardian. Which sensors move together and whether same-type sensors in one zone are independent are still to be measured on the real Twin [`docs/TWIN-HANDOVER.md`]; each is a switch that is built and waits for the answer.

Built to connect to a team it has never met

In Phase 2 the triplets are reshuffled. A new Twin or Guardian means a new adapter or a changed configuration file, never a change to the decision logic.

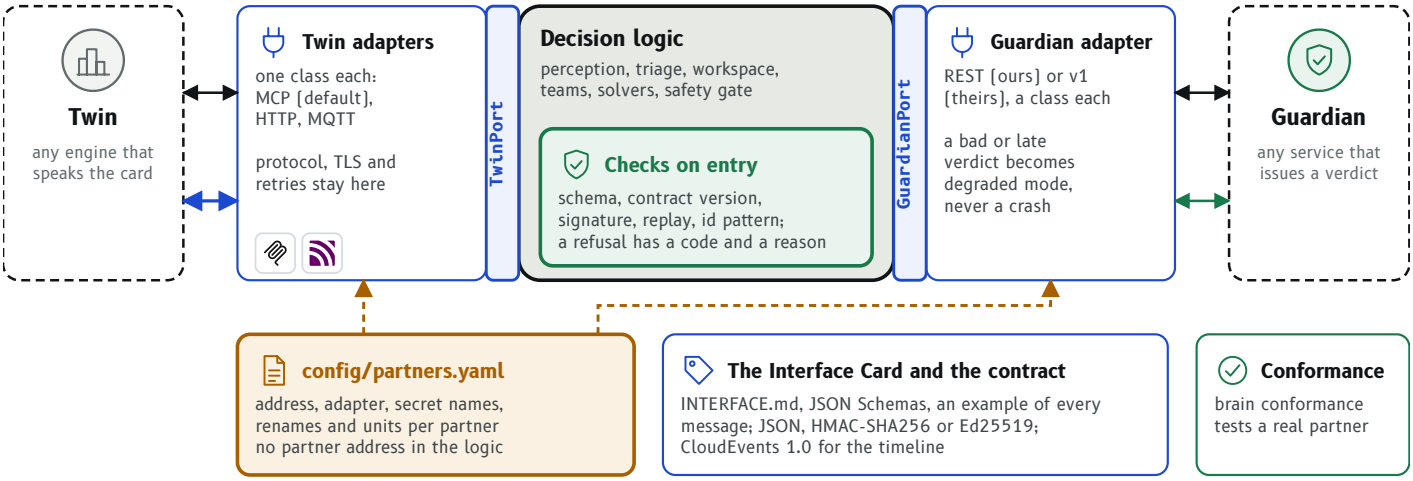


Figure 2c. Everything outside the two ports can be replaced; the logic cannot tell which partner it talks to.

The five things the jury looks for, and where to check each

Requirement	What the repository does	How to check it
Logic separate from partner I/O	The logic reaches partners only through `TwinPort` and `GuardianPort` (`ports.py`). Protocols live in `adapters/`, one class each: MCP, HTTP and MQTT for the Twin, REST for the Guardian.	`tests/test_layering.py` fails if logic imports an adapter or the reverse; `tests/test_twin_mqtt.py` runs `t14` over MQTT.
A complete Interface Card	`INTERFACE.md`: what the Brain receives and sends, an example of each message, how to repoint it, how it authenticates. Long form: `docs/INTEGRATION-brain.md`.	`tests/test_contracts.py` validates every example in `contracts/examples/` and verifies each signature.
Settings in configuration	`config/partners.yaml` holds each partner's adapter, address, secret variable name, renames and units. Secrets come from the environment; code defaults are loopback addresses.	A test requires the YAML to equal the defaults; `brain onboard` proposes edits for a renamed Twin.
Standard protocols and formats	MCP, REST and MQTT; JSON Schema; HMAC-SHA256 or Ed25519 over canonical JSON; Server-Sent Events and CloudEvents 1.0 described by AsyncAPI; TLS; OpenTelemetry.	`contracts/schemas/`; `docs/asyncapi.yaml` (55 event types).
Validation and clear errors	Every reading is checked on entry; a refusal carries a code and a reason (below) and goes to the timeline. A bad verdict puts the Brain in degraded mode; a failing Twin tool is named in the error.	`brain conformance` checks a real partner; `brain redteam`: 24 attacks, all refused.

What a partner is told when it sends something unexpected

The partner sends	Code	The Brain's answer, as the code returns it
unknown contract version	`bad_contract_version`	contract_version '2025-01-01' is not supported; this Brain accepts ['2026-09-17']
a field missing (zone_id)	`schema_invalid`	reading envelope is invalid: zone_id: Field required
no signature	`signature_missing`	wl-01-01#1: provenance.hmac is required
a value changed in transit	`signature_invalid`	wl-01-01#1: provenance.hmac does not match the payload
the same reading twice	`replay_or_out_of_order`	wl-01-01: sequence_no 1 is not greater than the last seen (1)
a number sent as text	`value_invalid`	wl-01-01#1: value for type 'water_level' is invalid: level_m: Input should be a valid number, unable to parse string as a number

Table 2. Run through the Brain's own perception code when this report is built. Not shown yet: the Brain has run only against mocks of the Twin and the Guardian; the HTTP and MQTT adapters are agreed but not yet run against a partner.

2. The City Brain

What should we do?

2.1 Role and goals

The Brain watches the Twin's live readings, detects real incidents (crimes, fires, accidents, jams, floods, overloads), asks the Guardian whether each reading can be believed, decides the response and acts through the Twin's actuators. It never acts on a reading the Guardian has not vouched for, and it never acts except through one checked door, the **safety gate.**

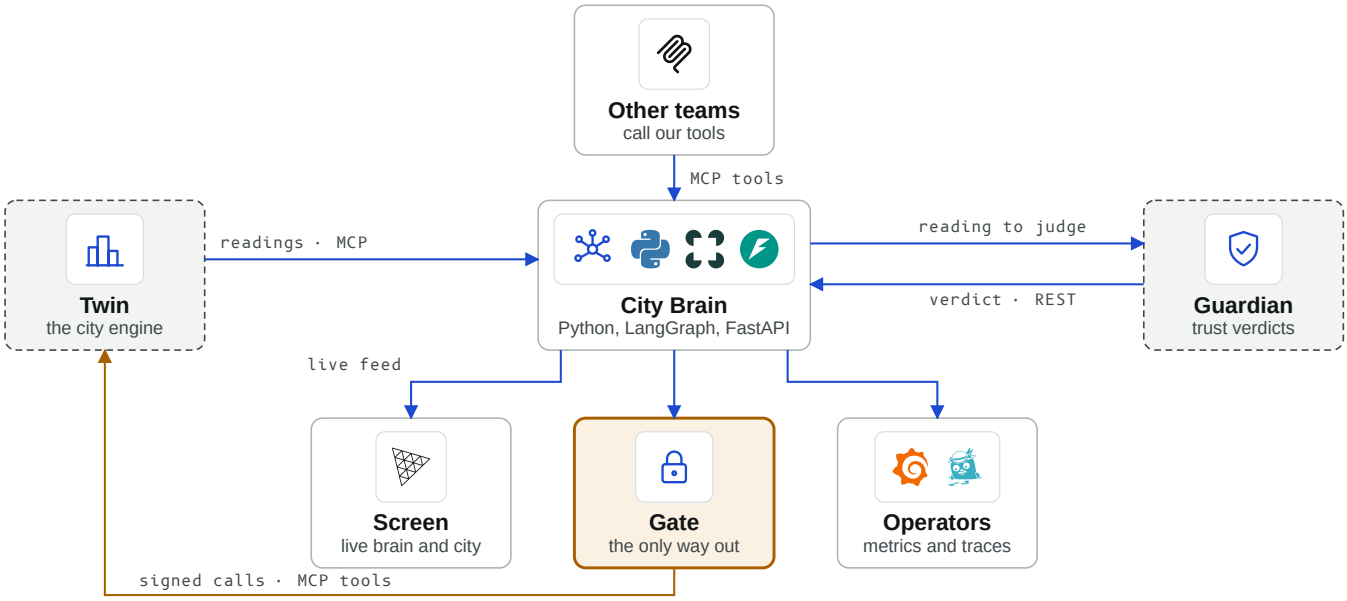


Figure 2.1. The Brain between its partners, one protocol on each link. Dashed cards are the other teams' layers; the amber card is the gate, the only path to the city.

Trust before action. Every actuator call names a trusted Guardian verdict that covers its evidence (`trust_ref`).

One way out. Rules, agents and solvers only propose; the gate checks, signs and sends.

Code disposes. A model may reason, code verifies; a scripted team passes every scenario with no network.

Nothing is silent. A held action, a refused reading, an outage: each is a named event with its reason.

5 / 5 Test Book scenarios T12–T16 pass, one chained log each	**3.5 ms** reflex answer after the evidence (t14)	**1,112** tests, named as sentences	**24 / 24** red-team attacks refused	**60 / 60** held-out variants decide as their base	**19** conformance checks a partner can run

Six departments, one per kind of incident

Department	Incidents	Acts through	Reasoning
Police	crime	`dispatch_unit` (pursuit, contain), `broadcast_alert`, `recall_unit`	team: Analyst, Tactician, Chief
Fire, Ambulance	fire, accident	`dispatch_unit`, `broadcast_alert`	desk: Analyst, Chief
Traffic control	jam, ambulance route	`set_signal_phase` (hold green)	desk
Disaster response	flood	`broadcast_alert`, `set_valve_state` (a person approves)	desk
Utilities	overload, smog	`broadcast_alert`, `set_grid_connection` (a person approves)	desk

A department is three files (a manifest of detectors and actuators, reflex rules, a playbook), so a new kind of incident is a new pack, not new code.

2.2 Architecture

One reading travels eighteen steps from the Twin to an action and an explanation. Each step is a small part with one job that hands its result to the next; each card names the technology that runs it and, in blue, the brain function it is modelled on (brain-inspired engineering, not a model of a brain).

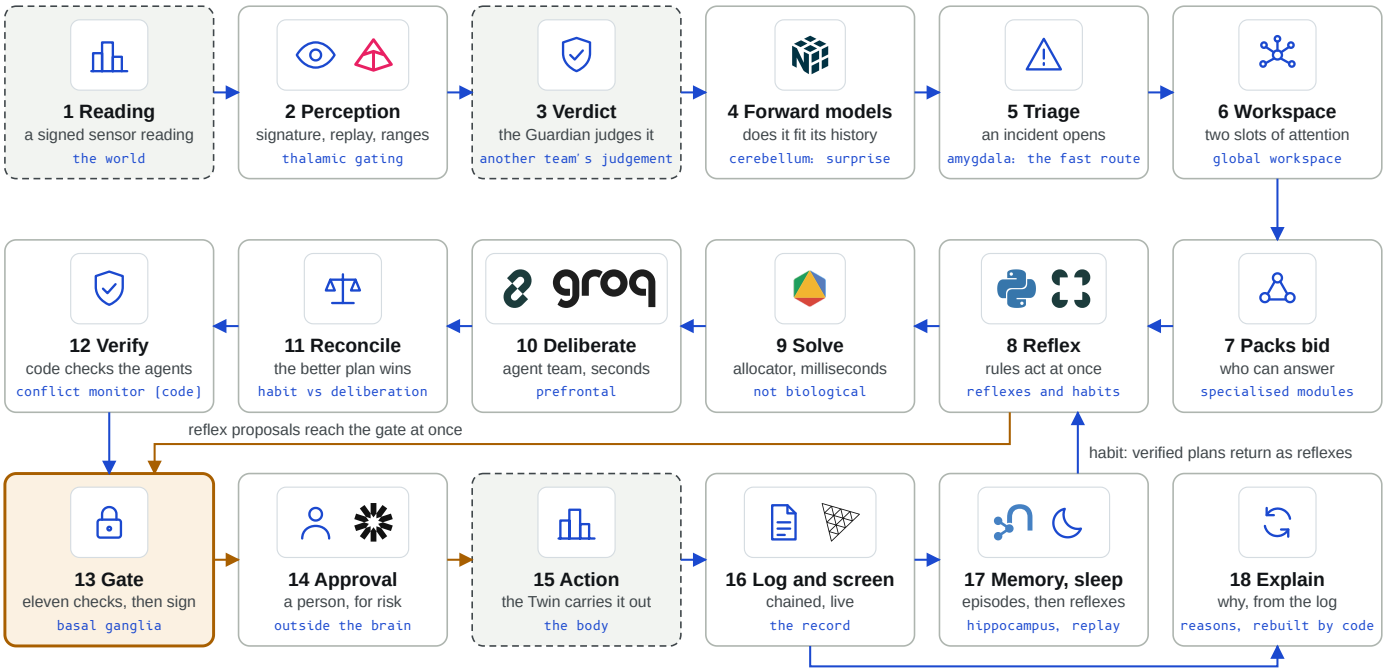


Figure 2.2. From a reading to a decision. Amber arrows are the way out: reflex proposals reach the gate at once, the team's plan follows and meets the same gate. A plan the team makes the same way twice returns as a reflex after sleep.

Incident detection

Perception checks the contract version, the HMAC signature, the sequence number (replay) and value ranges; a refusal has a code. **The Guardian's verdict** (2 s deadline) decides whether the reading may count. **Forward models** learn each sensor's own normal from trusted readings only: a reading many spreads away is a *surprise* and the reflex it would trigger is held until a second source agrees; a Kalman tracker forecasts when a river reaches the flood line. **Triage** runs the six departments' detectors to open or join an incident, and the **workspace** gives two seats of attention by severity, trust, surprise and urgency.

Instead of staying silent the Brain says `brain.human.needed` for an alarm no department knows, an incident with no procedure, or an incident that stalls.

Decision method: rules, agents and solvers, joined

Rules (13, in six packs) answer in milliseconds with no model. **Agent teams** weigh the case: for Police an Analyst, a Tactician and a Chief on LangGraph (Groq models with `--llm`, a scripted stand-in otherwise or when the model fails); the other departments use a two-role desk. **Solvers** compute what judgement should not guess (OR-Tools min-cost flow for dispatch, CP-SAT for a perimeter and for load shedding, Dijkstra for a green corridor, Kalman for the flood forecast). They run in code before the agents and their answers are written into the brief, so a model that never calls a tool still reads them.

Code then verifies the agents' plan (`verify_case`, `verify_plan`, `verify_desk`) before it reaches the gate.

What can stop a reading on its way to an incident, and what is said

The reading	Event on the timeline	Shown by
has a bad signature, a replayed sequence number or a value out of range	`brain.reading.rejected` with a stable code	`brain redteam`, `brain conformance`
is judged untrusted by the Guardian	`brain.action.blocked` (trust), nothing is sent	T13, T15
jumps further than its own history explains	`brain.percept.surprise`, the reflex is held	`brain forward-bench`

Two speeds, one gate. In T14 the reflex dispatches the first unit 3.5 ms after the evidence; the team's solver-checked perimeter follows about 70 ms later and passes the same eleven checks.

2.2 Architecture (continued): how the Brain picks an actuator

Three steps: the incident type decides which department may act and with which actuator; a solver or the nearest free station picks the device; the gate decides whether the call may go. The Brain never calls the Twin from anywhere else.

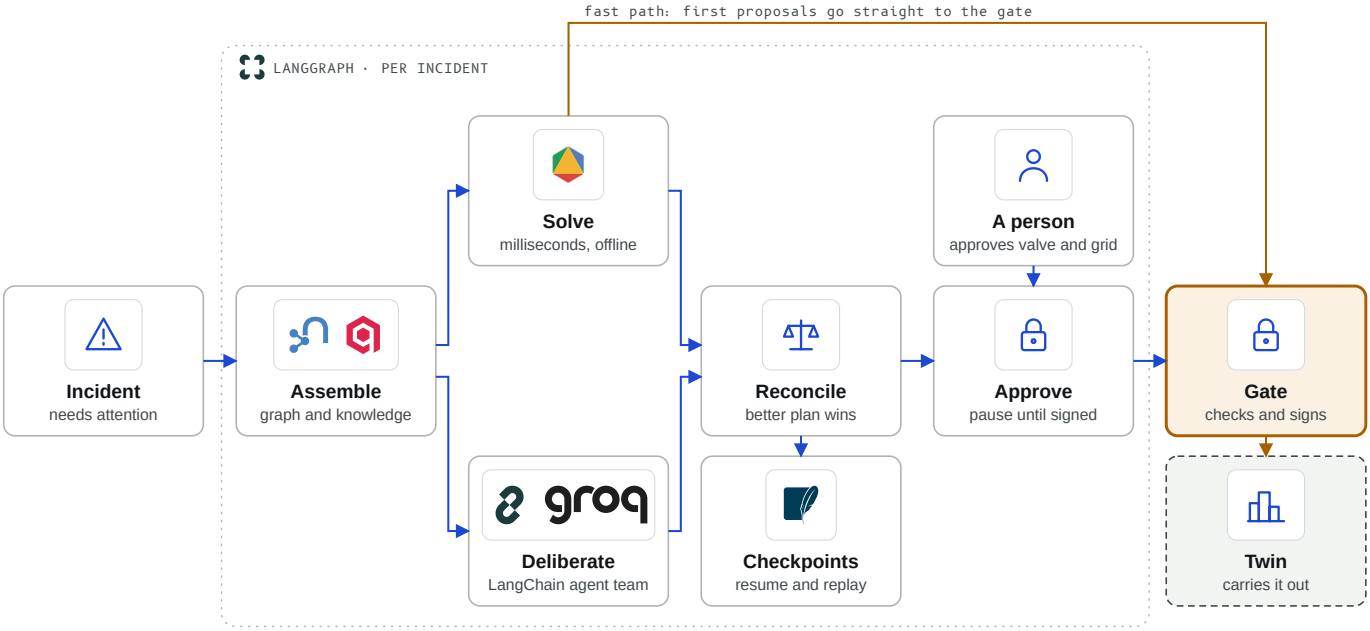


Figure 2.3. The planning graph for one incident (LangGraph): solve and deliberate in parallel, keep the better plan, pause for a person on a valve or the grid, checkpoint to SQLite so a run resumes and replays.

Which station or device

Nearest free station by hop distance, never a quarantined or reserved one (scenario T16: the Guardian quarantines a station, the gate refuses it with reason `quarantine` and another is sent). When two incidents want the same kind of unit, the dispatch solver solves them together and its choice is used only if it saves cost: for two collisions the far ambulance goes to the first so the near one is free for the second (2.8 against 4.2).

Which cut, which signal

A cascade analysis weighs what each grid cut would darken (a hospital counts 10) and the shed solver picks the cheapest set that removes the excess. A corridor holds the signals on an ambulance's route green. A dispatched unit's `unit_status` readings close the incident; an arrest recalls every unit still out.

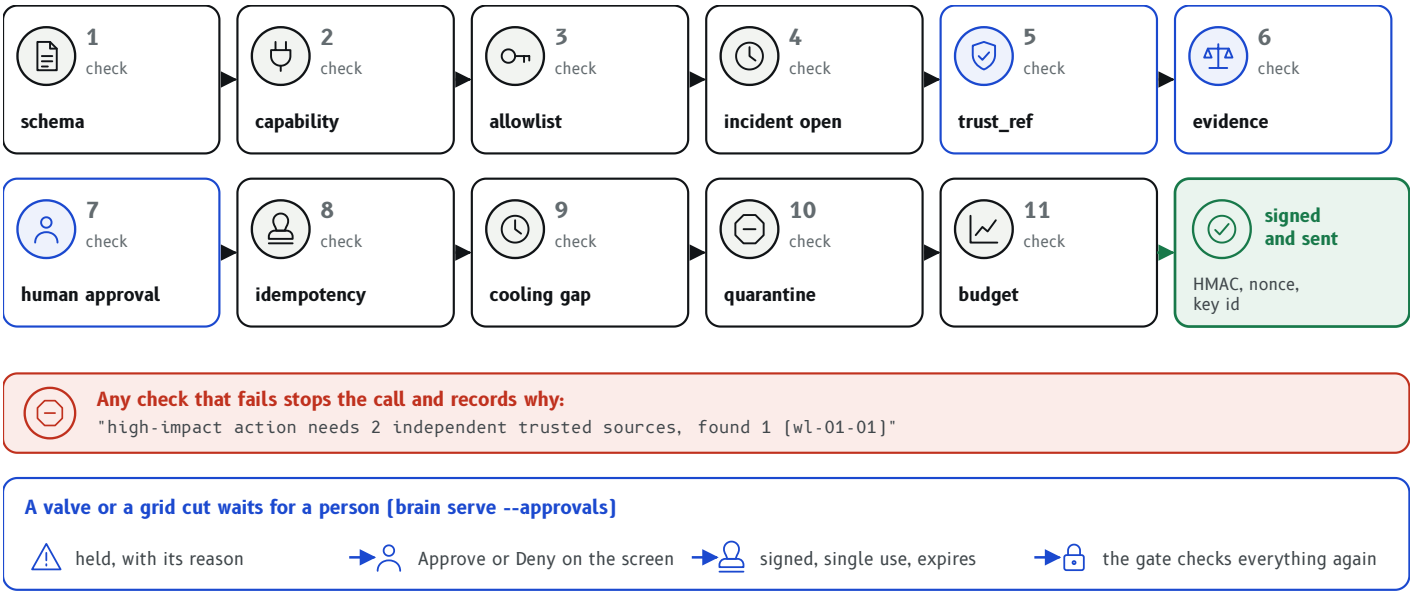


Figure 2.4. The safety gate. Every proposal, from a rule, an agent or a solver, meets the same eleven checks in order; the first that fails stops the call and is logged with its reason. A repeated call returns `duplicate` from the Twin.

2.3 Tech stack and why

Standard, well-known parts, each with one job. The stack runs offline on one laptop: the embedding model is baked into the image, the screen loads nothing from the internet, and a model outage only changes who reasons.

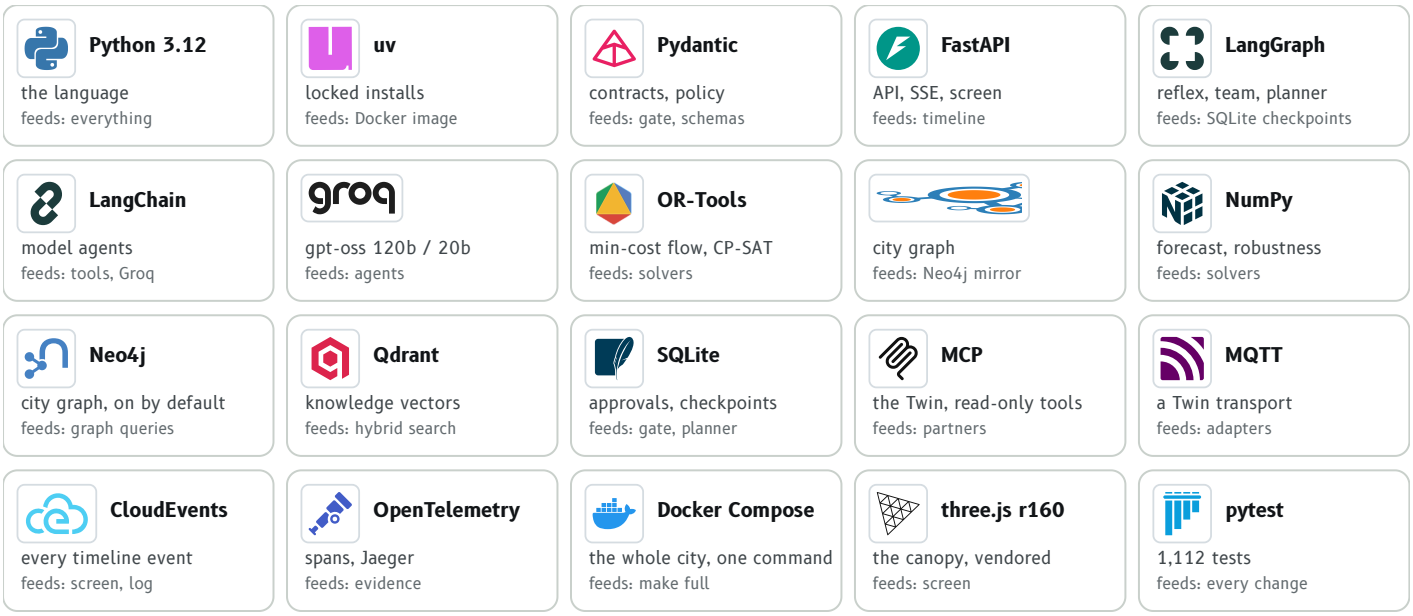


Figure 2.5. The stack and what each part feeds. Logos identify the technologies; each belongs to its owner.

Choice	Why
Python 3.12, uv, Pydantic	One language for rules, agents and solvers; locked, reproducible builds; every message a typed model that validates itself.
LangGraph + OR-Tools	Graphs that pause, resume and checkpoint for agents; proven solvers with a certificate for what must be exact.
MCP, REST, SSE + CloudEvents	MCP gives the Twin's tools schemas and discovery; REST is one question, one answer for verdicts; SSE is one-way, replayable, and needs no client library.
Neo4j + Qdrant	The city as a graph that answers "why was this unit sent?" in one query; hybrid search over 39 procedures. Decisions never wait on either.

2.4 Interfaces in/out

Direction	Message	Transport (default first) and contract
In, from the Twin	signed sensor readings; the city layout	MCP `list_readings(cursor)` every 200 ms and `get_topology()`; also plain HTTP or MQTT
In, from the Guardian	a signed trust verdict per reading; its containment list	with the Guardian team's contract 1.0: REST `POST /v1/verdicts/request` (by sensor and sequence number), HMAC checked; `GET /v1/containment`; `X-API-Key`. The Brain asks; it does not read the alert feed `WS /v1/ws/verdicts`. Our mock speaks a simpler REST contract (`POST /verdicts`)
Out, to the Twin	signed calls: `dispatch_unit`, `broadcast_alert`, `recall_unit`, `set_signal_phase`, `set_valve_state`, `set_grid_connection`	valve, grid, signal and dispatch go through the Guardian's audited `POST /v1/actuator-calls`; broadcast and recall, which its registry does not list, go to the Twin directly (MCP tool of the same name, or HTTP)
Out, to anyone	timeline events; the why of a decision; read-only tools	SSE `GET /events` (CloudEvents 1.0, 55 types in AsyncAPI); `/api/decisions/{id}/explanation`; MCP at `/mcp`

All of it is in the one-page Interface Card (`brain/INTERFACE.md`) with a JSON example of every message and JSON Schemas in `contracts/`. A new partner is an edit of `config/partners.yaml` (address, adapter, secret names, renames, units), never a code change: decision code reaches partners only through two ports, and `tests/test_layering.py` fails the build if it imports a protocol.

2.5 Security measures

Every input is hostile until checked, every model answer is a suggestion, and the only way out is one signed call that a person approves when it is high-impact. Threats are mapped to MITRE ATT&CK for ICS and to MITRE ATLAS for the AI parts [`docs/SECURITY-MITRE.md`].

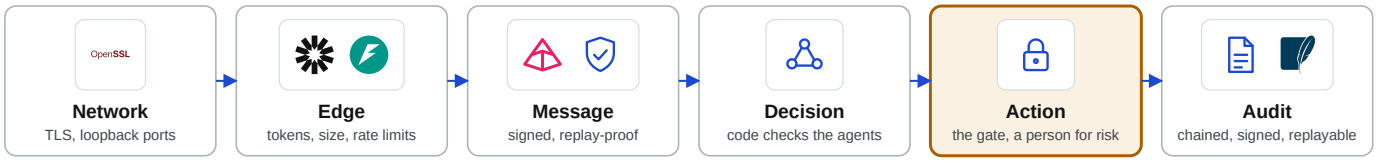


Figure 2.6. Six layers, each assuming the one outside it can fail.

Boundary	Control
Twin to Brain: readings	HMAC-SHA256 or Ed25519 over canonical JSON, key ids with expiry, a rising sequence number per sensor, schema and ranges; only allowlisted fields are kept, so demographic fields never reach a decision
Guardian to Brain: verdicts	schema check; the gate asks for more independent trusted sources as impact rises; only the Guardian's token may retract a verdict; the 1.0 adapter checks the verdict's HMAC
People, partners to the API	scoped bearer tokens off loopback [read, push, approve, admin], body and rate limits; a partner may push only `twin.*` or `guardian.*` events, a forged `brain.*` is refused
Brain to the language model	ids limited to safe characters before any prompt; read-only tools, no actuating tool exists; models pick zone ids, never free text; code verifies; then the gate
Brain to Twin: calls	eleven gate checks; each call signed with its time and a one-time nonce; a valve or grid call needs a signed approval bound to its exact parameters, used once
Brain to its own disk	hash-chained log with a signed head; learning only from trusted readings; self-tuning limited to a listed set of numbers, each with a range

MITRE technique	Attack run against the code	Result
ICS T1692.002 forged reading	wrong key, no signature, a field edited after signing, a replayed sequence number, `alg: none`	refused every time
ICS T1692.001, T0849 forged call	an approval reused with changed parameters; the Twin pushing as the Guardian	refused
Enterprise T1190, T1078	no token, a forged token, a read token doing admin work, 28 path spellings by 7 methods	none got through
ATLAS AML.T0051, T0053	30 prompt-injection strings in sensor ids and values; a tool that tries to actuate	dropped before any prompt; no such tool can exist

Evidence. `brain redteam`: 24 attacks, all refused [re-run for this report]. A partner's bad message is answered with a stable code and a sentence [`signature_invalid`, `replay_or_out_of_order`, `value_invalid`…] and shown on the timeline. A silent Guardian gives no verdict, so nothing it has not judged is acted on [T15]: the Guardian's fail-safe rule, enforced on the Brain's side too.

2.6 Known limitations

- **No joint run with the real Twin yet.** End to end only against mocks; against the live Guardian, the fake and real flood and T16 [`logs/t16-live-guardian-*.log`].
- **The mock Guardian is scripted.** T13 and T16 show what the Brain does *given* a verdict, not whether a real Guardian would give it.
- **Our mock's contract does not sign verdicts.** The Guardian team's 1.0 does, and the Brain checks it.
- **Tuned on stand-ins.** Which sensors move together, and whether same-zone sensors are independent, wait for the real Twin's data [`docs/TWIN-HANDOVER.md`].
- **TLS and the freshness limit are off by default.** Signatures still protect readings and calls; set `twin.max_age_s` on the live Twin.
- **One dispatch per incident at the Guardian.** Its auditor refuses a second dispatch for one incident [`duplicate_dispatch`], so T14's perimeter units would be refused. To agree: a duplicate is the same incident *and* station.
- **Guardian alerts are not read.** An actuator it flags unreliable reaches the Brain through the containment list, which the gate obeys; `compromised_twin` does not.

4. Test Book (the Brain's scenarios)

Five scenarios, each one trigger (`scenarios/triggers/tN.sh` or `uv run brain scenario tN`) and one hash-chained log in `logs/`, with the expected outputs in the scenario file. Together they cover the brief's four interaction rows; the Guardian team's T1 to T11 are in the shared `TESTBOOK.md`. The generated list, in the book's fields, is `brain/docs/TESTBOOK-brain.md`.

ID	Scenario	Interaction	Expected output
T12	A calm day	Twin → Brain	no incident, no actuator call: the Brain does not overreact
T13	A fake flood	Twin → Guardian, Brain → Guardian	the gauge is untrusted; the gate refuses the warning and the valve call; nothing reaches the city
T14	A crime and a pursuit	Twin → Brain, Brain → Guardian	reflex dispatch, a team perimeter checked and verified, units recalled at the arrest
T15	A flood, an attack and a layer down	all three	warning sent, valve held, forged report sends no police; once the Guardian is silent, one 'Guardian away' and no action on unjudged readings
T16	A quarantined police station	Brain → Guardian	the gate refuses calls to the quarantined station with reason `quarantine`; a different station is used